여러분은,

치팅시트와 기억이 다를 때,
무엇을 믿으시나요?

AI 내부 신호로
기억과 문맥이 맞는지 구분하는 연구

이화여자대학교 박세령 · 이다영 · 손현경

CONTENTS

발표 목차

01	기억과 치팅시트	무엇을 믿을까요?	04–16
02	내부 신호와 헤드	AI 내부에서 상태를 알아낼 수 있을까요?	17–29
03	본선에서 확인한 성능	새 질문에서도 상태를 구분할까요?	30–37
04	적은 자료로 정예팀 찾기	주니어도 정예팀을 잘 뽑을까요?	38–42
05	활용과 계획	읽은 신호를 어디에 쓸 수 있을까요?	43–50

연구 요약 03 Live demo 47 APPENDIX 52–60

2분 시연을 포함한 10분 발표. 본문 50장 / 보충 자료 8장 / 구분·종료 2장.

AI 내부 신호로 4가지 상태를 구분했습니다
학습 자료를 줄여도 성능은 비슷했습니다

연구 문제

기억 K와 치팅시트 C가 서로 다른 답을 줄 수 있습니다.
어느 정보를 믿을지 정하려면, 각각 맞는지부터 구분해야 합니다.

K × C
4가지 정답 상태

제안 방법

도움이 되는 헤드를 고르고, 작은 분류기인 Probe로 상태를 예측했습니다.
612개 질문에 맞는 문맥과 틀린 문맥을 붙여 검증했습니다.

1,224
질문과 문맥의 조합

핵심 결과

선발한 헤드만 사용해도 전체 표현과 성능이 비슷했습니다.
예산 ¼로 학습한 경우에도 평균 Macro-F1은 0.6120이었습니다.

예산 ¼
평균 Macro-F1 차이 −0.0181

후속 방향

답이 나오기 전에 상태를 읽고, 남은 계산에 그 결과를 반영하려 합니다.
올바른 정보를 따라 답하도록 유도하는 것이 다음 목표입니다.

one-pass
같은 입력 처리 중 읽고 반영

자료: E14·E16 실험 결과, 교수님 미팅 자료 (12). 80%·20% 비교는 평균값 기준.

01

기억과 치팅시트

무엇을 믿을까요?

01 기억과 치팅시트
02 내부 신호와 헤드
03 본선에서 확인한 성능
04 적은 자료로 정예팀 찾기
05 활용과 계획

AI가 올바른 정보를 따라 답하도록 돕고자 합니다

이미 알고 있던 정보와 새로 받은 문서가 다를 때,

기억과 문맥이 각각 맞는지 구분해,
올바른 쪽으로 답변을 유도하려 합니다.

이번에 확인한 것
두 정보가 맞는지 구분

→

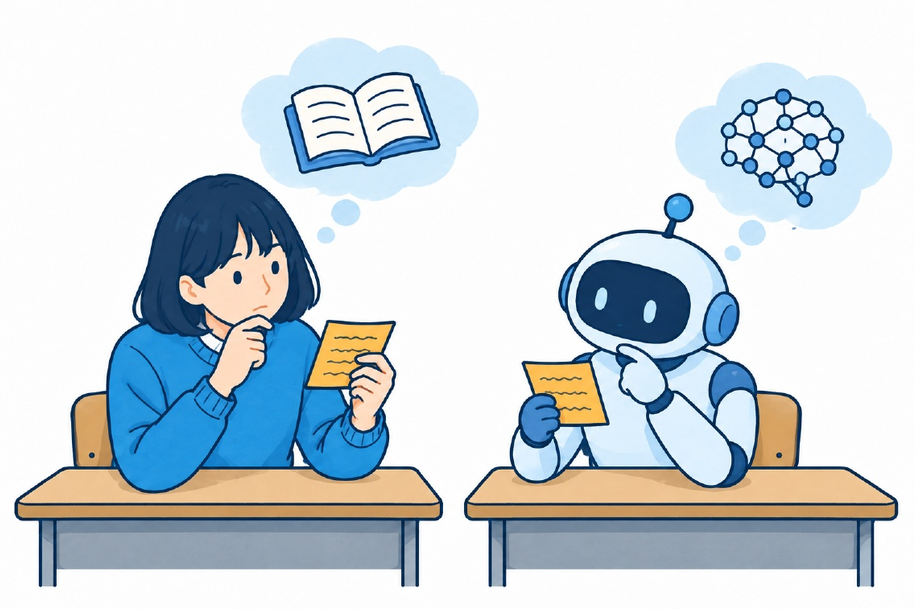
다음에 할 일
믿을 정보 선택

→

답변 생성에 반영
틀린 답을 고치기

이번에는 내부 신호로 상태를 구분하는 방법을 확인했습니다. 이 결과로 실제 답변을 바꾸는 방법은 다음 단계에서 검증합니다.

LLM을 치팅시트를 받은 학생에 비유해볼까요?



학생은 기억과 치팅시트로 답을 씁니다.
LLM은 기존 지식과 문맥으로 답을 만듭니다.

문제를 푸는 학생	→	**LLM** 답을 생성하는 모델
공부한 기억	→	**K · 기존 지식** 모델이 학습으로 익힌 정보
시험장의 치팅시트	→	**C · 새로 받은 문맥** 질문과 함께 받은 검색 결과·문서
학생의 답안지	→	**AI의 최종 답변** 기존 지식과 문맥을 이용해 만든 답

LLM은 많은 텍스트를 학습해 답을 생성하는 모델입니다. 사람과의 대응은 설명을 위한 비유입니다.

치팅시트라고 다 맞는 건 아니죠

"

전날 졸면서 쓰다가 틀렸을 수도 있잖아요.

기억도 틀릴 수 있고,
치팅시트도 틀릴 수 있습니다.

시험장에 왔는데, 두 답이 다릅니다

문제

상관계수가 0이면,
두 변수는 항상 독립일까?

내 기억에는

항상 그렇지는
않다

치팅시트에는

항상
독립이다

틀린 치팅시트를 따르면,
원래 풀 수 있던 문제도 틀립니다.

통계학 개념을 이용한 설명용 가상 상황입니다. 실험 데이터나 실제 모델의 답변 예시가 아닙니다.

시험장에 왔는데, 두 답이 다릅니다

문제

상관계수가 0C
두 변수는 항상

내 기억에는

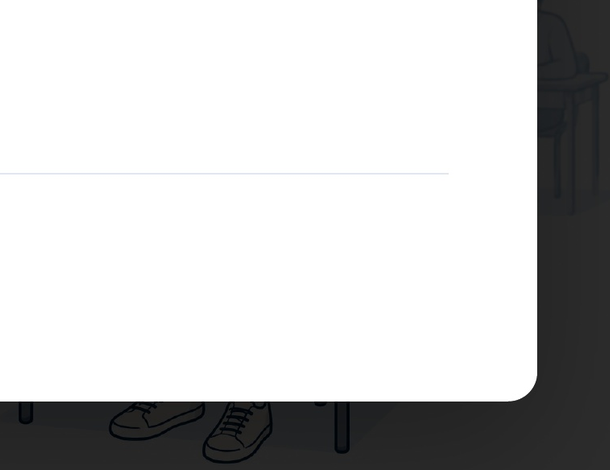
항상 그렇지는
않다

조교님께 물어볼 수 있다면?

조교님,

내 기억과 치팅시트 중
뭐가 맞는지 알려주실 수 있나요?

상태를 알면,
어떤 정보를 믿어야 할지도 판단할 수 있습니다.



틀린 치팅시트를 따르면,
원래 풀 수 있던 문제도 틀립니다.

통계학 개념을 이용한 설명용 가상 상황입니다. 실험 데이터나 실제 모델의 답변 예시가 아닙니다.

기억을 믿을 때와 치팅시트를 믿을 때

기억을 믿을 때

기억 ✓ 치팅시트 ×

기억의 답을 채택합니다

기억이 맞고 치팅시트가 틀렸다면,
기억에 근거해 답합니다.

치팅시트를 믿을 때

기억 × 치팅시트 ✓

치팅시트의 답을 채택합니다

기억이 틀리고 치팅시트가 맞다면,
치팅시트에 근거해 답합니다.

조교님이 구분해줘야 할 상황은 4가지입니다

	기억이 맞음 K+	기억이 틀림 K−
C+ 치팅시트 맞음	K+C+ **두 정보 모두 채택 가능**	K−C+ **치팅시트를 채택**
C− 치팅시트 틀림	K+C− **기억을 채택**	K−C− **둘 다 신뢰할 수 없음**

어떤 정보를 믿을지는 상태에 달려 있습니다.

기억과 치팅시트가 맞고 틀릴 때를 조합하면 4가지 상태가 됩니다.

K는 문맥 없이 답한 결과, C는 제공된 문맥의 정답 여부입니다. 강조 장면의 대화는 이해를 위한 가상 예시입니다.

조교님이 구분해줘야 할 상황은 4가지입니다

	기억이 맞음 K+	기억이 틀림 K−
C+ 치팅시트 맞음	K+C+ **두 정보 모두 채택 가능**	K−C+ **치팅시트를 채택**
C− 치팅시트 틀림	K+C− **기억을 채택**	K−C− **둘 다 신뢰할 수 없음**

상태 1 · K+C+

상관계수 0이면 항상 독립일까?

기억 맞음

"항상 독립인 것은 아니다."

치팅시트 맞음

"상관이 0이어도 의존할 수 있다."

두 정보가 같은 정답을 가리킵니다.

K는 문맥 없이 답한 결과, C는 제공된 문맥의 정답 여부입니다. 강조 장면의 대화는 이해를 위한 가상 예시입니다.

조교님이 구분해줘야 할 상황은 4가지입니다

	기억이 맞음 K+	기억이 틀림 K−
C+ 치팅시트 맞음	K+C+ **두 정보 모두 채택 가능**	K−C+ **치팅시트를 채택**
C− 치팅시트 틀림	K+C− **기억을 채택**	K−C− **둘 다 신뢰할 수 없음**

상태 2 · K+C−

상관계수 0이면 항상 독립일까?

기억 맞음

"항상 독립인 것은 아니다."

치팅시트 틀림

"상관이 0이면 항상 독립이다."

이때는 기억의 답을 채택합니다.

K는 문맥 없이 답한 결과, C는 제공된 문맥의 정답 여부입니다. 강조 장면의 대화는 이해를 위한 가상 예시입니다.

조교님이 구분해줘야 할 상황은 4가지입니다

	기억이 맞음 K+	기억이 틀림 K−
C+ 치팅시트 맞음	K+C+ **두 정보 모두 채택 가능**	K−C+ **치팅시트를 채택**
C− 치팅시트 틀림	K+C− **기억을 채택**	K−C− **둘 다 신뢰할 수 없음**

상태 3 · K−C+

상관계수 0이면 항상 독립일까?

기억 틀림

"상관이 0이면 항상 독립이다."

치팅시트 맞음

"상관이 0이어도 의존할 수 있다."

이때는 치팅시트의 답을 채택합니다.

K는 문맥 없이 답한 결과, C는 제공된 문맥의 정답 여부입니다. 강조 장면의 대화는 이해를 위한 가상 예시입니다.

조교님이 구분해줘야 할 상황은 4가지입니다

	기억이 맞음 K+	기억이 틀림 K−
C+ 치팅시트 맞음	K+C+ **두 정보 모두 채택 가능**	K−C+ **치팅시트를 채택**
C− 치팅시트 틀림	K+C− **기억을 채택**	K−C− **둘 다 신뢰할 수 없음**

상태 4 · K−C−

상관계수 0이면 항상 독립일까?

기억 틀림

"상관이 0이면 항상 독립이다."

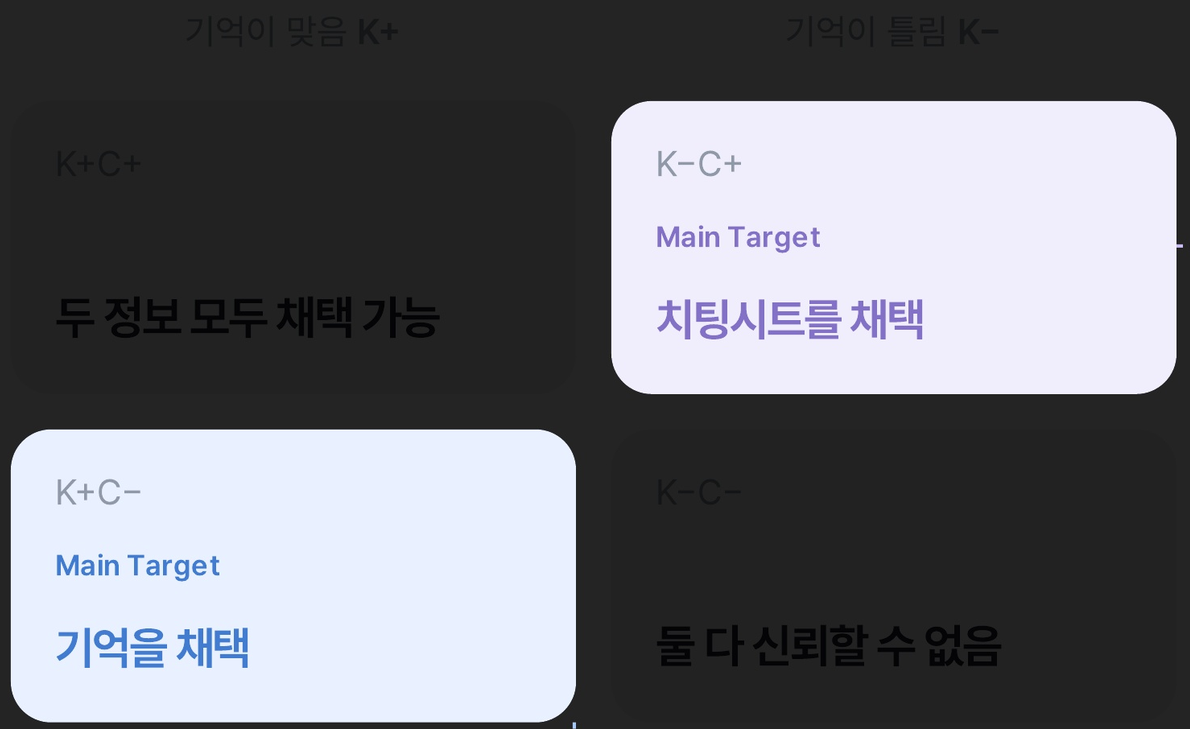
치팅시트 틀림

"상관이 0이면 항상 독립이다."

두 답이 같아도, 둘 다 틀릴 수 있습니다.
추가 근거가 필요합니다.

K는 문맥 없이 답한 결과, C는 제공된 문맥의 정답 여부입니다. 강조 장면의 대화는 이해를 위한 가상 예시입니다.

조교님이 구분해줘야 할 상황은 4가지입니다



Why Main Target?

잘못된 쪽을 따르면,
맞힐 수 있는 답도 틀립니다.

한쪽은 맞고 다른 쪽은 틀리므로,
어느 정보를 채택하느냐가 중요합니다.

K+ C− **기억만 맞을 때**
틀린 치팅시트에 흔들리지 않게

K− C+ **치팅시트만 맞을 때**
틀린 기억을 고집하지 않게

핵심 대상은 K+ C−와 K− C+입니다. 두 상태의 구분을 통해 올바른 정보의 채택을 돕는 것이 목표입니다.

02

내부 신호와 헤드

AI 내부에서 상태를 알아낼 수 있을까요?

01 기억과 치팅시트
02 내부 신호와 헤드
03 본선에서 확인한 성능
04 적은 자료로 정예팀 찾기
05 활용과 계획

LLM은 중간 연산 과정을 확인해 볼 수 있습니다

사람의 경우

머릿속의 판단을
그대로 꺼내 보기는 어렵습니다.

말과 행동을 보고
무엇을 믿는지 추측합니다.

LLM의 경우

답을 만드는 도중에
계산한 숫자를 볼 수 있습니다.

이 숫자를 내부 신호로 삼아,
기억과 문맥이 맞는지 추정합니다.

우리는 LLM의 내부 신호를 읽어서, 이런 조교 역할을 구현해보려 합니다.

사람과의 비교는 설명을 위한 비유입니다. 내부 숫자가 정답을 직접 알려 주지는 않으므로, 별도로 학습한 Probe로 상태를 추정합니다.

내부 신호로 4가지 상태를 구분하는 방법을 만들었습니다

이번에 한 일

내부 신호로 상태 구분

도움이 되는 헤드를 고르고
작은 분류기로 상태를 예측합니다.

방법을 만들고 실험으로 확인

앞으로 할 일

올바른 답으로 유도

어느 정보를 믿을지 정하고
남은 계산에 반영하려 합니다.

후속 연구

상태를 잘 구분하는지와 실제 답변이 개선되는지는 각각 평가합니다.

LLM은 앞 단계의 결과를 이어받아 계산합니다

각 계산 단계를 **Layer(레이어)**라고 부릅니다.

중간에 상태를 알아내면, 남은 계산을 올바른 쪽으로 유도할 수 있습니다.

Llama 3.1 8B는 32개 Layer를 순차적으로 통과합니다. L0–L31은 0-based 층 번호입니다.

LLM은 앞 단계의 결과를 이어받아 계산합니다

우리가 만들고 싶은 변화

K+ C− · 기억이 맞고 문맥이 틀린 상황

기억을 따르도록 남은 계산을 유도한다면?

잘못된 문맥을 따른 답

"상관이 0이면
항상 독립이다."

→

올바른 정보를 따른 답

"상관이 0이어도
독립이 아닐 수 있다."

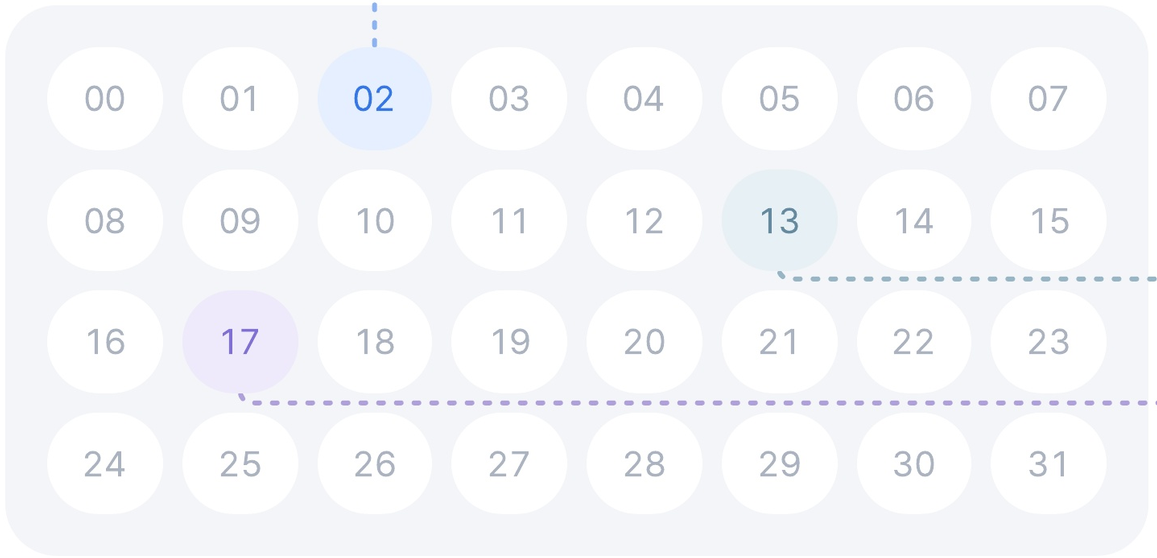
기억을 채택하도록 유도해, 올바른 답을 생성하는 것이 목표입니다.

상태 구분은 실험으로 확인 · 답변 개선은 다음에 검증

Llama 3.1 8B는 32개 Layer를 순차적으로 통과합니다. L0–L31은 0-based 층 번호입니다.

어텐션 안에는 정보를 모으는 헤드가 있습니다

사용한 모델은 각 Layer에 어텐션 헤드가 32개 있습니다



이화여대 해커톤 참가자에 빗대면

1 학생 1
특징: AI 파트
모델을 만들고 예측을 담당

2 학생 2
특징: 백엔드 파트
서버와 데이터를 연결

3 학생 3
특징: 프론트엔드 파트
사용자가 보는 화면을 구성

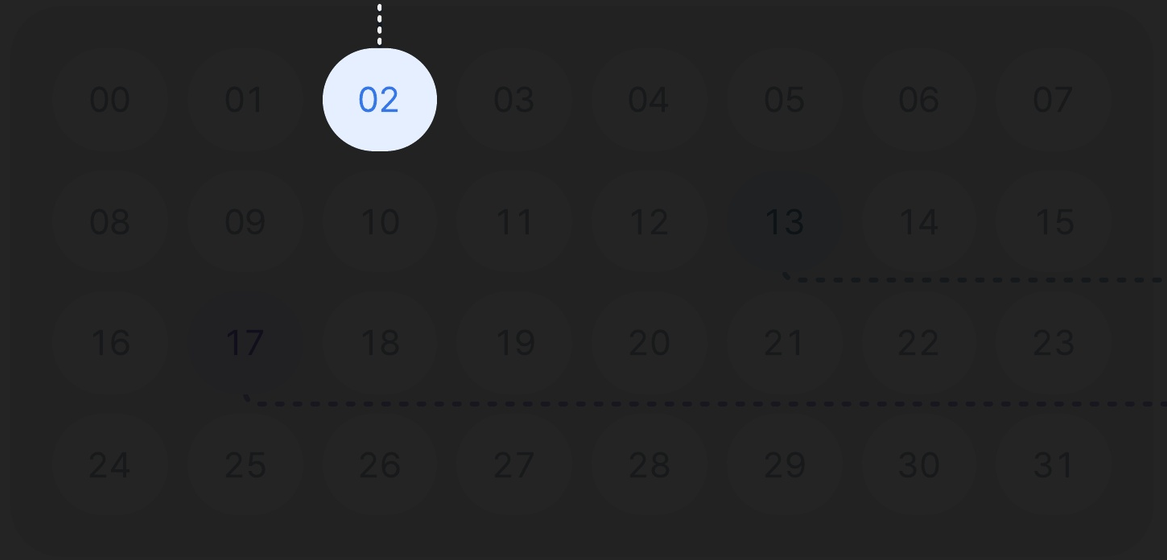
32개 헤드의 정보에 다른 계산을 더하면, **한 Layer의 최종 결과인 '전체 표현'이 됩니다.**

모델 구조와 전체 표현 보기 ↗ Appendix

학생의 특징은 역할 차이를 설명하는 비유입니다. 예시 헤드 번호가 실제로 해당 파트를 맡는다는 뜻은 아닙니다.

어텐션 안에는 정보를 모으는 헤드가 있습니다

사용한 모델은 각 Layer에 어텐션 헤드가 32개 있습니다



이화여대 해커톤 참가자에 빗대면

학생 1 · 특징: AI 파트

모델을 만들고,
어떤 예측을 내놓을지 담당합니다.

학생 2

특징: 백엔드 파트

서버와 데이터를 연결

학생 3

특징: 프론트엔드 파트

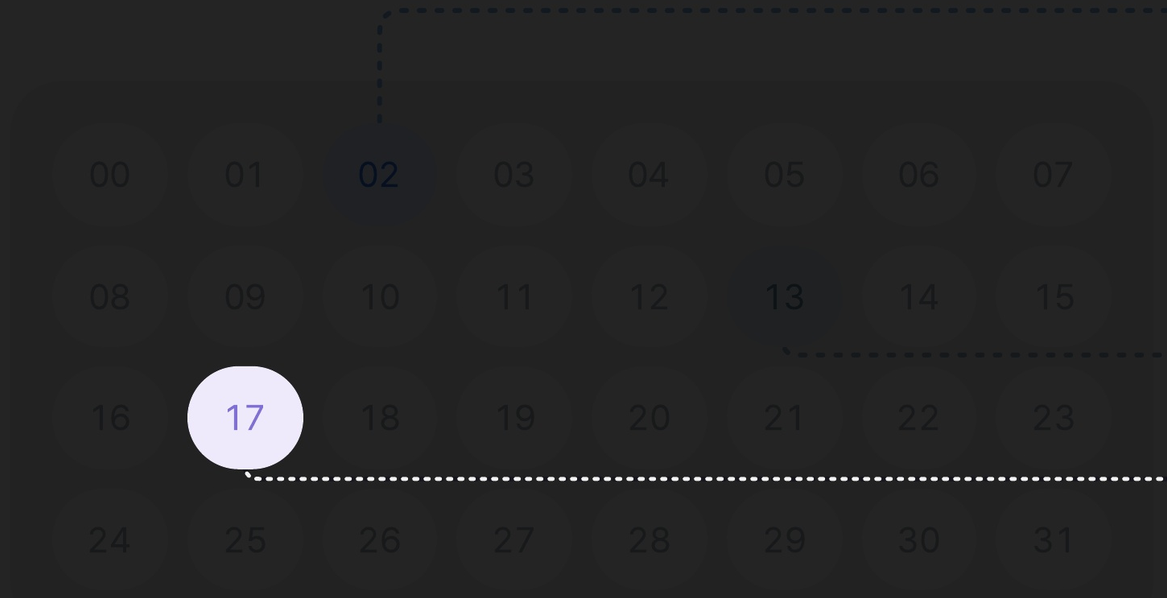
사용자가 보는 화면을 구성

각 학생에게 잘하는 일이 있듯, **헤드도 서로 다른 정보를 모아 다음 계산에 보탭니다.**

학생의 특징은 역할 차이를 설명하는 비유입니다. 예시 헤드 번호가 실제로 해당 파트를 맡는다는 뜻은 아닙니다.

어텐션 안에는 정보를 모으는 헤드가 있습니다

사용한 모델은 각 Layer에 어텐션 헤드가 32개 있습니다



이화여대 해커톤 참가자에 빗대면

학생 1
1
특징: AI 파트
모델을 만들고 예측을 담당

학생 2 · 특징: 백엔드 파트

서버와 데이터를 연결해,
서비스가 작동하도록 만듭니다.

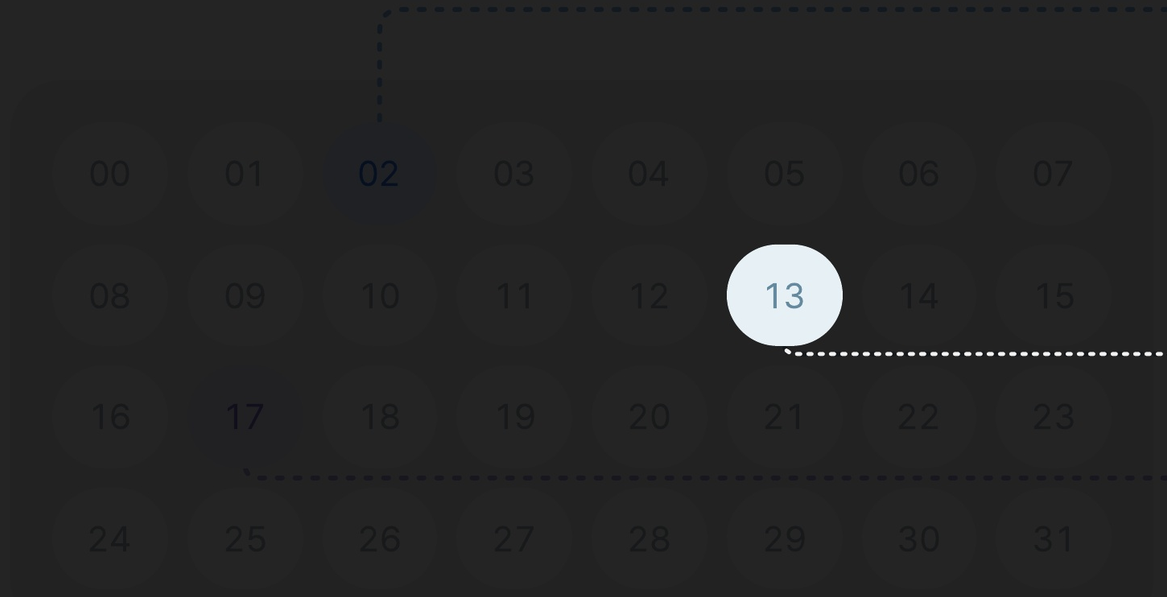
학생 3
3
특징: 프론트엔드 파트
사용자가 보는 화면을 구성

각 학생에게 잘하는 일이 있듯, **헤드도 서로 다른 정보를 모아 다음 계산에 보탭니다.**

학생의 특징은 역할 차이를 설명하는 비유입니다. 예시 헤드 번호가 실제로 해당 파트를 맡는다는 뜻은 아닙니다.

어텐션 안에는 정보를 모으는 헤드가 있습니다

사용한 모델은 각 Layer에 어텐션 헤드가 32개 있습니다



이화여대 해커톤 참가자에 빗대면

학생 1

1 특징: AI 파트

모델을 만들고 예측을 담당

학생 2

2 특징: 백엔드 파트

서버와 데이터를 연결

학생 3 · 특징: 프론트엔드 파트

사용자가 보는 화면을 만들고,
상호작용을 구현합니다.

각 학생에게 잘하는 일이 있듯, **헤드도 서로 다른 정보를 모아 다음 계산에 보탭니다.**

학생의 특징은 역할 차이를 설명하는 비유입니다. 예시 헤드 번호가 실제로 해당 파트를 맡는다는 뜻은 아닙니다.

이화여대 해커톤, 예선으로 정예팀을 꾸려볼까요?

시니어 심사위원

💰 💰 💰 💰

이번 해커톤의 과제

기억과 치팅시트의 4가지 상태를 구분하기

32명의 작업 결과 중
어떤 신호가 도움이 될까요?

01 **예선 · 32명의 작업 결과를 평가**

학생의 작업 결과는 각 헤드의 내부 신호에 해당합니다.

02 **팀빌딩 · 과제에 맞는 조합을 선발**

Probe의 검증 점수로 유용한 헤드 조합을 고릅니다.

03 **본선 · 처음 보는 문제로 실력 확인**

학습과 선발에 쓰지 않은 질문으로 평가합니다.

조교 역할에 필요한 신호를 찾기 위해, Probe를 예선 평가 도구로 사용합니다.

해커톤은 설명용 비유입니다. Probe는 헤드 신호로 상태를 예측하는 선형 분류기이며, 검증 점수로 헤드 조합을 선발합니다.

예선 평가는 정답이 있는 예시에서 시작합니다

01 예선 준비 · 구분 기준 학습

**학생의 작업 결과와 정답을 보고
어떻게 평가할지 배웁니다.**

학생의 결과 · 헤드 신호 + 실제 상태 · 라벨

Probe는 헤드 신호와 정답 상태를 함께 보고,
4가지 상태를 구분하는 기준을 배웁니다.

02 예선 평가 · 헤드 조합 비교

**다른 예선 문제에서도
상태를 잘 구분하는지 봅니다.**

검증 질문의 헤드 신호 → 상태 예측

학습에 쓰지 않은 검증 질문으로 비교합니다.
이번 과제에 유용한 조합을 정예팀으로 고릅니다.

본선 문제는 따로 남깁니다. 본선 정답은 예선 학습이나 팀 선발에 쓰지 않습니다.

학습·검증·최종 평가 질문을 나눕니다. 헤드 선발에는 검증 점수를 쓰며, 최종 평가 질문의 라벨은 선발이나 학습에 사용하지 않습니다.

예선의 평가 도구, Probe는 작은 분류기입니다

정답이 있는 예시로 배운 **상태 구분 기준**을 담고 있습니다.

검증 문제에서 나온

헤드 신호

h_1 h_2 h_3 ...

헤드가 계산한
숫자들을 입력합니다.

예선에서 학습한 평가 도구

학습된 Probe

신호에 가중치를 적용해
4가지 상태의 점수를 계산

어떤 신호를 얼마나 볼지,
정답이 있는 예시로 배웠습니다.

점수가 가장 높은

상태를 예측

K+C+	K−C+
K+C−	K−C−

이 예시에서는
치팅시트만 맞는 상태로 분류

헤드 조합마다 Probe로 상태를 예측하고, 검증 점수가 높은 조합을 선발합니다.

선형 분류기의 작동 방식을 설명한 개념도입니다. 강조한 상태는 설명용 예시이며 실측 예측 결과가 아닙니다.

32개 헤드 중, 정예 후보 4개가 반복해서 뽑혔습니다

반복해서 찾은 후보 4개 · 32개 중 12.5%

헤드 신호를 직접 비교해, 문맥·충돌 판단에 쓸 후보를 찾았습니다.

어텐션이 잘 보는 것

(학생이 잘하는 것)

H20	문맥 정답 여부 · 충돌 여부	5 회 / 5회 평가
H10	문맥 정답 여부	5 회 / 5회 평가
H19	문맥 정답 여부	5 회 / 5회 평가
H3	충돌 여부	4 회 / 5회 평가

E14 역할별 반복 선택 후보입니다. 4-state 판독에 사용한 최종 헤드 집합은 분할마다 달라지며, 1~32개를 사용했습니다.

03

본선에서 확인한 성능

처음 보는 본선 문제에서도 잘할까요?

01 기억과 치팅시트

02 내부 신호와 헤드

03 본선에서 확인한 성능

04 적은 자료로 정예팀 찾기

05 활용과 계획

본선은 예선에서 보지 못한 실제 질문으로 평가합니다

정예팀과 Probe를 확정한 뒤, 본선 문제의 상태를 예측하고 정답과 비교합니다.

612

개의 지식 질문

C+
맞는 문맥

C−
틀린 문맥

→

1,224

개의 질문·문맥 조합

612개 질문을 예선과 본선으로 나누고,
각 질문에는 문맥 2종을 붙였습니다.

같은 데이터셋을 질문 단위로 나눠 예선·본선에 사용합니다. 한 질문의 C+·C−는 같은 분할에 두며, 본선 라벨은 학습·선발에 쓰지 않습니다.

F1은 정밀도와 재현율을 함께 보는 점수입니다

지표 이해를 위한 가상 예시

예를 들어, '기억만 맞는 상태'를 찾는다면?

정밀도 · 찾아낸 것 중 맞은 비율

60%

그 상태라고 예측한 10건 중
실제로도 그 상태인 것은 6건

재현율 · 실제 대상 중 찾은 비율

75%

실제로 그 상태인 8건 중
그 상태라고 찾아낸 것은 6건

F1 = 0.67 두 비율을 함께 반영합니다.

10건·8건·6건은 지표 설명용 가상 수치입니다. F1은 정밀도와 재현율의 조화평균입니다. 정의: scikit-learn f1_score.

Macro-F1은 4가지 상태의 F1을 평균한 점수입니다

4가지 상태의 F1을 각각 계산한 뒤 평균냅니다

K+C+	K+C−	K−C+	K−C−
¼	¼	¼	¼
이 상태의 F1	이 상태의 F1	이 상태의 F1	이 상태의 F1

상태별 F1의 평균 **Macro-F1**

개수가 적은 상태도 같은 비중으로 평가합니다. Macro-F1 0.63은 답변 정답률 63%를 뜻하지 않습니다.

검증 03 본선에서 확인한 성능

4가지 상태를 무작위로 고르면, 점수는 약 0.25입니다

단서 없이 4가지 상태를 같은 확률로 고릅니다.

K+C+	K+C−	K−C+	K−C−
25%	25%	25%	25%

각 상태를 25% 확률로 고르면, 평균적으로 ¼을 맞힙니다.

우리 데이터의 무작위 Macro-F1

0.2477

약 0.25를 비교 기준으로 사용합니다

같은 정답 라벨로 20,000회 반복 계산

정답 상태 수 362 / 362 / 250 / 250. F1은 상태별 비중의 영향도 받습니다. 균등 무작위 평균이며, 매번 0.25 아래라는 뜻은 아닙니다.

효과 03 본선에서 확인한 성능

본선에서, 정예팀의 F1이 전체 표현보다 높았습니다

전체 표현이란?

모든 헤드의 정보가 합쳐진 결과

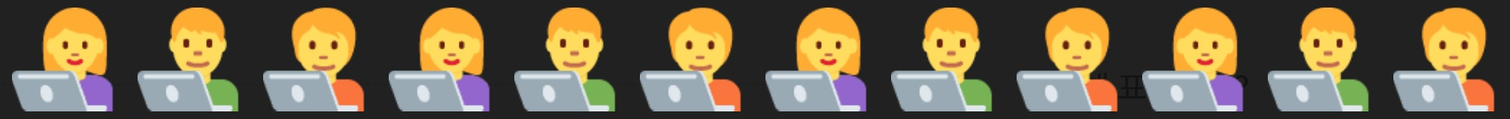
32개 헤드의 정보에
다른 계산도 더한,
Layer의 최종 출력입니다.

전체 표현 0.6215 → 선발 조합 0.6306
F1 스코어가 0.0091 높았습니다.

E14 pooled OOF. 정예팀은 분할마다 선발한 헤드 조합(1~32개)입니다. 반복 후보 네 개를 고정해 낸 점수는 아닙니다.

본선에서, 정예팀의 F1이 전체 표현보다 높았습니다

전체 표현이란?



전체 학생이 함께 개발한 결과

=

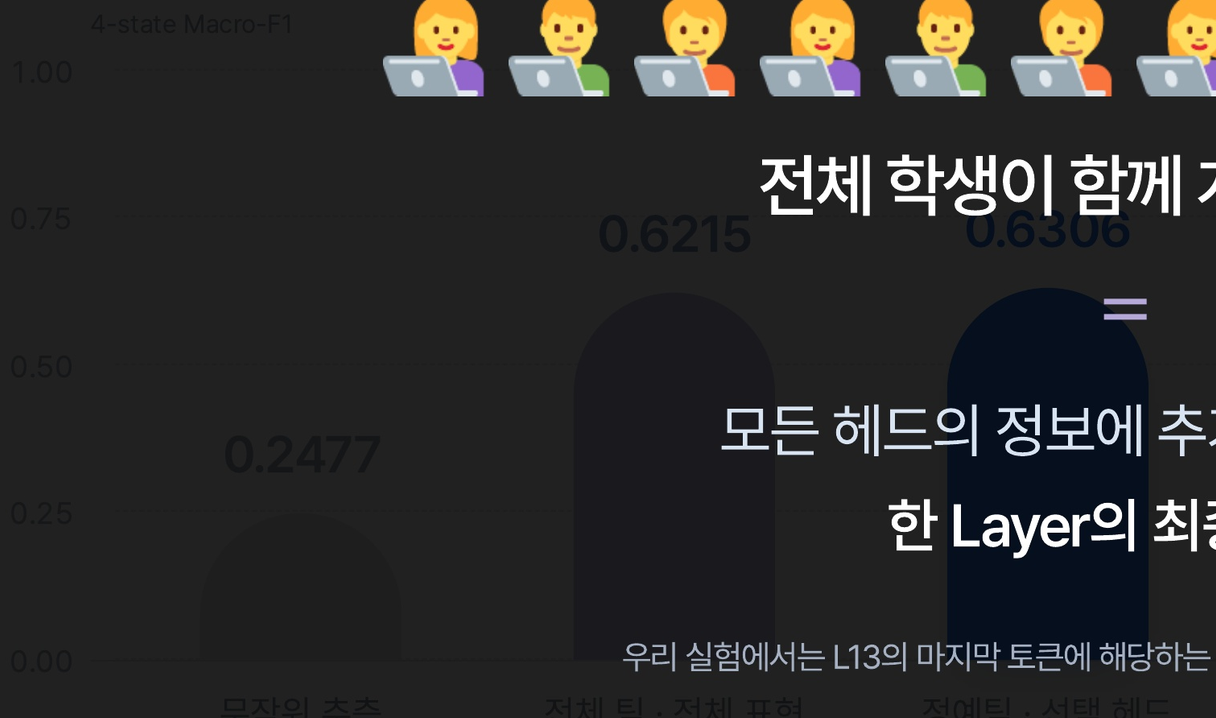
모든 헤드의 정보에 추가 연산을 더한

한 Layer의 최종 출력

모든 헤드의 정보가 합쳐진 결과

32개 헤드의 정보에
추가 연산도 더한,
Layer의 최종 출력입니다.

전체 표현 0.6215 → 선발 조합 0.6306
F1 스코어가 0.0091 높았습니다.



우리 실험에서는 L13의 마지막 토큰에 해당하는 4,096차원 표현을 사용했습니다.

E14 pooled OOF. 정예팀은 분할마다 선발한 헤드 조합(1~32개)입니다. 반복 후보 네 개를 고정해 낸 점수는 아닙니다.

F1 스코어는 0.6306, K·C의 AUROC도 확인했습니다

4가지 상태를 함께 분류

0.6306

4-state Macro-F1

각 상태의 F1을 구한 뒤,
4개 값을 같은 비중으로 평균냅니다.

맞는 사례에 더 높은 점수를 줄까요?

기억 K의 정답 여부
K AUROC

0.8322

K+와 K−를 하나씩 뽑아 비교하면,
K+에 더 높은 점수를 줄 가능성이 약 **83.2%**

치팅시트 C의 정답 여부
C AUROC

0.9225

C+와 C−를 하나씩 뽑아 비교하면,
C+에 더 높은 점수를 줄 가능성이 약 **92.3%**

AUROC는 0.5면 무작위 수준, 1이면 맞는 사례를 항상 더 높게 평가합니다.

E14 pooled OOF. AUROC는 사례 쌍의 점수 순위를 평가하며, 같은 점수는 0.5회로 계산합니다. 답변 정답률을 뜻하지 않습니다.

04

적은 자료로 정예팀 찾기

주니어도 정예팀을 잘 뽑을까요?

01 기억과 치팅시트
02 내부 신호와 헤드
03 본선에서 확인한 성능
04 적은 자료로 정예팀 찾기
05 활용과 계획

긴축재정! 예선 평가를 주니어에게 맡겨도 될까요?

주니어가 배울 예선 자료

20%

정답 상태를 알려 줄 질문은
전체의 20%입니다.

122
정답을 알려 줄 예선 질문

490
따로 남긴 본선 질문

시니어 심사위원

💰 💰 💰 💰

약 80% 라벨로 학습

예산 축소 →

주니어 심사위원

💰

20% 라벨로 학습

그래도 정예팀을 잘 뽑을까요?

시니어·주니어는 학습 자료량의 차이를 설명하는 비유입니다. 20% 라벨로 헤드를 선택하고 Probe를 학습한 뒤, 남긴 질문으로 본선 평가를 합니다.

주니어가 선발한 팀의 본선 F1은 평균 0.6120입니다

주니어 정예팀 · 본선 성적

0.61

20% 자료로 헤드를 고르고 Probe를 학습한 결과

20% 라벨 조건
5회 평균 · 표준편차 0.012

같은 490개 본선 질문으로 세 조건을 평가

E16 supervised Probe 결과. seed 42~46의 5회 반복 평균이며, 각 seed 안에서는 세 라벨 조건에 같은 평가 질문을 사용했습니다.

시니어와 주니어가 선발한 팀, 본선 점수 차이는 0.0181

시니어 vs 주니어

1/4

75% 적은 예산

평균 점수 차이 **−0.0181**

평균값 기준: E14 5-fold 0.6301 ± 0.0472 / E16 5회 반복 0.6120 ± 0.0123.

예선 자료를 줄여도, 본선 성능은 비슷했습니다

01 전체 팀의 결과를 사용

전체 표현

32개 헤드의 정보와
다른 계산을 합친 결과

0.6215

본선 Macro-F1

약 80% 라벨

02 시니어가 정예팀을 선발

선발한 헤드 조합

점수가 좋은 헤드를 골라
4가지 상태를 구분

0.6301

본선 Macro-F1

시니어 · 약 80% 라벨

03 주니어가 정예팀을 선발

예산 ¼

전체 질문의 20%만 보고
헤드 선택과 학습을 진행

0.6120

본선 Macro-F1

주니어 · 20% 라벨

정예 후보 4/32 = 12.5%

H20 · H10 · H19 · H3

예산 ¼ = 25%

75% 적은 예산 · 평균 점수 차이 0.0181

12.5%는 32개 중 반복 후보 4개의 비율입니다. 위 점수를 낸 조합은 분할별 1~32개입니다. 25%는 시니어 대비 학습 자료 예산이며, 전체 계산 비용 비율이 아닙니다.

05

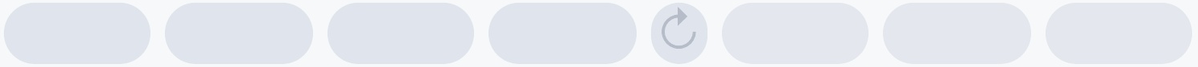
활용과 계획

읽은 신호를 어디에 쓸 수 있을까요?

01 기억과 치팅시트
02 내부 신호와 헤드
03 본선에서 확인한 성능
04 적은 자료로 정예팀 찾기
05 활용과 계획

답을 만드는 도중에, 올바른 쪽으로 유도할 수 있다면?

답안지를 낸 뒤
“다시 풀어 봐”



답을 확인하고
처음부터 다시 풀기

풀이 중간에
“치팅시트를 다시 봐”

남은 풀이에서
올바른 정보 따르기

one-pass: 한 번의 입력 처리 중 상태를 읽고, 남은 계산에 반영하는 방식

같은 prefill 안의 read → write 구조입니다. 이미 지난 레이어로 돌아가거나, 이후 계산을 생략한다는 뜻은 아닙니다.

상태를 읽고 답변에 반영하려면, 너무 이르지도 늦지도 않은 Layer가 필요합니다

상태를 읽고, 뒤쪽 계산에 반영합니다

L7 먼저 상태 읽기 → **L8–12** 뒤쪽 계산에 반영

L13 상태 다시 읽기 → **L14–18** 뒤쪽 계산에 반영

상태를 읽은 뒤, 다음 Layer의 계산에 반영합니다.

너무 이르거나 늦으면 왜 어려울까요?

너무 이르면

먼저 문제를 읽고 이해할 시간이 필요합니다.
LLM도 문맥을 충분히 처리해야
상태를 구분할 단서가 생깁니다.

너무 늦으면

조교에게 힌트를 받은 뒤에도,
답을 고치고 마킹할 시간이 필요합니다.
LLM도 이를 반영할 계산이 남아 있어야 합니다.

KEY POINT 상태를 구분할 단서가 충분하고, 이후 계산에 개입할 여유가 있어야 합니다.

같은 입력 처리 안에서 상태를 읽은 뒤, 이후 Layer에 개입하는 구조입니다. 층 번호는 0-based입니다.

상태를 읽고 답변에 반영하려면, 너무 이르지도 늦지도 않은 Layer가 필요합니다

상태를 읽

너무 이르면

먼저 문제를 읽고 이해할 시간이 필요합니다.

LLM도 문맥을 충분히 처리해야 상태를 구분할 단서가 생깁니다.

너무 늦으면

조교의 힌트를 받은 뒤, 답을 고치고 마킹할 시간도 필요합니다.

LLM도 이를 반영할 뒤쪽 계산이 남아 있어야 합니다.

단서가 충분하고, 답을 고칠 여유가 있는 시점에 개입합니다.

상태를 읽은 뒤, 다음 Layer의 계산에 반영합니다.

답을 고치고 마킹할 시간이 필요합니다.

있어야 합니다.

KEY POINT 상태를 구분할 단서가 충분하고, 이후 계산에 개입할 여유가 있어야 합니다.

같은 입력 처리 안에서 상태를 읽은 뒤, 이후 Layer에 개입하는 구조입니다. 층 번호는 0-based입니다.

시연: 본선 문제의 상태를 예측해 보겠습니다

02:00

미리 수집한 실제 헤드 신호로
Probe를 실행합니다.

A

예선 · 20% 자료로 헤드 선택과 학습

학습·평가 질문 수와 선택한 헤드 확인

B

본선 · 처음 보는 질문의 상태 예측

예측과 실제 상태를 비교하고, 틀린 사례도 확인

C

5·10·20% 라벨 조건 비교

이번 실행 결과와 앞선 5회 실험 결과 확인

실제 캐시는 발표 전에 준비합니다. 이 시연은 상태 구분이며, 온라인 one-pass steering 시연과는 구분합니다.

앞으로는 이 신호로 실제 답변을 바꿔 보겠습니다

연구 성과

- **4-state 분류 방법 구현**
  헤드 신호로 K/C 상태 구분 · Macro-F1 0.6306
- **저라벨 데이터로 Probe 학습**
  20% 라벨에서도 평균 Macro-F1 0.6120
- **Probe-free 추론용 헤드 조합 발견**
  추론 시 Probe 없이 판단 · one-pass 활용 후보

최종 목표

틀린 답은 고치고, 맞던 답은 지키기

10월 답변을 바꾸는 방법 실험

읽은 신호를 남은 계산에 반영
맞던 답이 틀려지는지도 확인

11월 새 데이터로 추가 평가

다른 질문에서도 통하는지 확인
답변의 정답률과 실행 시간 측정

헤드 선택에는 라벨을 사용했습니다. Probe-free는 추론 단계의 판단 방식이며, 해당 신호를 one-pass에 활용해 답변을 개선하는 것이 후속 목표입니다.

기억과 문맥이 충돌할 때, 선택할 근거를 찾습니다

AS-IS

지금의 문제

**기억과 문맥이 충돌하면,
무엇을 믿어야 할까요?**

K 기억 ≠ C 치팅시트

↓

어느 쪽이 올바를까?

잘못된 정보를 따르면
맞힐 수 있는 답도 틀립니다.

OUR METHOD

이번에 확인한 것

**헤드 신호를 읽어
4가지 상태를 구분합니다.**

헤드 신호 → Probe

K+ C+	K− C+
K+ C−	K− C−

선발한 헤드로 비슷한 성능,
예산 ¼에서도 평균 **0.6120**

TO-BE

이어 갈 목표

**읽은 상태를 이용해
올바른 선택을 돕습니다.**

K+ C− 기억을 채택하도록

K− C+ 치팅시트를 채택하도록

틀린 답은 고치고,
맞던 답은 지키기

실제 답변이 개선되는지는
다음 실험에서 확인합니다.

이번에 확인한 것은 상태 구분 성능입니다. 실제 답변 개선은 후속 목표이며, 0.6120은 상태 분류의 평균 Macro-F1입니다.

기억과 문맥이 충돌할 때, 선택할 근거를 찾습니다

AS-IS
지금의 문제

OUR METHOD
이번에 확인한 것

TO-BE
이어 갈 목표

Core Value

필요한 헤드 신호를 골라,
적은 자료로도 상태를 구분합니다.

이 단서로 AI가 **올바른 정보를 선택하도록** 돕겠습니다.

기억과 문맥이 충돌하면,
무엇을 믿어야 할까요?

K 기억 ≠ C 치팅시트

어느 쪽이 올바를까?

잘못된 정보를 따르면
맞힐 수 있는 답도 틀립니다.

헤드 신호를 읽어
[illegible] 상태를 구분합니다.

K+ C− K− C+ K− C−

내부 신호로 4가지 상태 구분 · 예산 ¼에서도 Macro-F1 0.6120

선발한 헤드로 비슷한 성능,
예산 ¼에서도 평균 **0.6120**

읽은 상태를 이용해
올바른 선택을 돕습니다.

K+ C− 기억을 채택하도록

K− C+ 치팅시트를 채택하도록

틀린 답은 고치고,
맞던 답은 지키기

실제 답변이 개선되는지는
다음 실험에서 확인합니다.

이번에 확인한 것은 상태 구분 성능입니다. 실제 답변 개선은 후속 목표이며, 0.6120은 상태 분류의 평균 Macro-F1입니다.

EOD

APPENDIX

질의응답을 위한 보충 자료

Llama 3.1 8B에서 헤드 신호와 전체 표현을 읽는 위치

모델 전체 흐름

질문 + 치팅시트

↓

토큰화 · 임베딩

↓

32개 Layer를 순서대로 통과

L0 ··· L12

L13 확대 →

L14 ··· L31

↓

최종 정규화 · 출력층

↓

다음 토큰 예측

한 Layer 안의 계산

이전 Layer의 출력

↓

RMSNorm

↓

Attention

H0 | H1 | ... | H31

32개 헤드의 출력

↓

출력 결합 · W_O

↓

이전 표현 더하기 +

↓

RMSNorm · MLP

↓

잔차 더하기 → Layer 최종 출력

헤드 신호

각 학생이 만든 결과

각 헤드가 모은 정보
헤드별 128차원

32개 query head · 8개 KV head

전체 표현

전체 학생이 만든 결과

모든 헤드의 정보에
MLP와 잔차 연결까지 반영

실험 기준: L13 · 마지막 토큰
4,096차원 BLOCK_LAST

↖ 22페이지 헤드 소개로 돌아가기

Meta Llama 3.1 8B 설정 및 Transformer 구조. 화살표는 계산 흐름을 단순화했으며, 본 실험의 전체 표현은 L13 block 출력입니다.

예선과 본선은 서로 다른 질문으로 나눕니다

예선 · 학습과 선발에 쓸 질문

헤드 선택과 Probe 학습

정답 상태를 알려 줍니다.

- 질문 A · C+ / C−
- 질문 B · C+ / C−
- 질문 C · C+ / C−

본선 · 처음 보는 평가 질문

예측이 끝난 뒤 실제 상태와 비교

평가 질문의 라벨은 학습과 선택에 쓰지 않습니다.

- 질문 D · C+ / C−
- 질문 E · C+ / C−
- 질문 F · C+ / C−

같은 질문의 두 문맥은 학습과 평가에 나누어 넣지 않습니다.

E14는 질문 단위 5-fold 검증입니다. A~F 표시는 분리 방식을 설명하는 예시입니다.

실험별 점수와 평가 조건

평가	설정	4-state Macro-F1
E14 전체 표현	5-fold pooled OOF 1,224개 입력	0.6215
E14 약 80% 개발 라벨	fold마다 개발 80% / 평가 20% 5-fold 평균 ± 표준편차	**0.6301 ± 0.0472** pooled OOF 0.6306
E16 20% 라벨	122질문 학습 / 490질문 평가 seed 42~46 평균 ± 표준편차	**0.6120 ± 0.0123**

80%와 20% 비교 장에서는 평균값 0.6301과 0.6120을 사용했습니다.

E14의 약 80%는 학습·후보 선택·검증을 포함한 개발 범위입니다. 판독 대상: E14 fold별 locked head-set / E16 소량 라벨로 선택한 role-head 벡터.

실험마다 선택한 헤드 수는 달랐습니다

fold 0	fold 1	fold 2	fold 3	fold 4
1	32	16	16	32

일부 분할에서는 전체 32개 헤드를 사용했습니다.

자주 뽑힌 헤드는 더 살펴볼 후보로 볼 수 있습니다.

이 헤드가 답을 바꾸는지, 적은 수만으로도 충분한지는 추가로 확인해야 합니다.

역할별 H20/H10/H19/H3 반복 선택과 4-state decoder의 헤드 선택은 별개의 분석입니다.

무작위 Macro-F1이 정확히 0.25가 아닌 이유

기준	우리 데이터의 값
4가지 상태를 각각 25% 확률로 추측	정답률 기댓값 **0.2500**
균등 무작위 Macro-F1, 20,000회	평균 **0.2477**
정답 상태별 입력 수	362 / 362 / 250 / 250

F1은 정답과 예측의 분포에도 영향을 받습니다.

유한 표본의 한 번의 추측은 0.25를 넘을 수 있습니다. 무학습 LLM의 실측 성능이 아니라 균등 무작위 기준입니다.

20% 라벨과 one-pass가 뜻하는 것

표현	정확한 범위
20% 라벨	헤드 선택·Probe 학습에 허용한 정답 상태표 평가 라벨은 예측 후 채점에 사용
one-pass	같은 prefill에서 읽고, 이후 레이어에 개입 토큰 생성과 오프라인 분석은 별도의 계산 필요
이번 노트북	사전 수집한 실제 헤드 신호로 CPU 판독 온라인 one-pass steering 실험과 구분

80%에서 20%로 줄이면 라벨 예산은 약 75% 감소합니다. LLM의 전체 계산 비용이 75% 줄었다는 뜻은 아닙니다. one-pass에서도 이후 Layer의 계산은 계속합니다.

Probe로 평가하고, 실제 실행은 헤드 신호로 판단합니다

학습한 Probe의 상태 구분	**0.6306 / 0.6120** E14 / E16 라벨 20%
Probe 없는 규칙의 상태 구분	**0.3860 / 0.3818** E14 / E16 라벨 20%
E14 실제 답변의 변화	**회복 2 · 손상 0 · 순증 2** 전체 1,224개 입력에서 작은 변화

앞으로는 내부 신호를 이용해, 올바른 정보를 따르는 답변을 늘리려 합니다.

E14: 분류 성능 평가는 학습한 Probe, 실제 실행은 헤드 신호와 고정 기준을 사용합니다. 0.6306은 runtime 규칙의 성능이 아닙니다.

F1·정확도·AUROC는 평가하는 대상이 다릅니다

지표	평가하는 것	E14 값
4-state Macro-F1	각 상태의 F1을 계산한 뒤 같은 비중으로 평균	0.6306
4-state 정확도	예측한 상태가 실제 상태와 일치한 비율	0.6422
K AUROC	기억이 맞는 사례에 더 높은 점수를 주는지 평가	0.8322
C AUROC	문맥이 맞는 사례에 더 높은 점수를 주는지 평가	0.9225

E14 locked head-set pooled OOF, 1,224개 입력. 5-fold 평균값과 pooled OOF 값을 구분합니다.